

Section 1 — Why Pydantic Exists

Concept Notes

What is Pydantic?

Pydantic is a Python library used for **data validation, data parsing, and type checking** using Python type hints.

In real-world applications, data can come from many different sources, such as:

- User input
- API requests
- JSON files
- Databases
- Forms
- Environment variables
- Configuration files
- External services
- AI model responses

External data cannot always be trusted. It may contain:

- Wrong data types
- Missing fields
- Invalid values
- Incorrect formats
- Unexpected extra data

Pydantic acts like a **data checkpoint** between external data and the main application.

The basic flow is:

External Data



Pydantic Validation



Valid Data



Application

If the data is invalid:

External Data



Pydantic Validation



Validation Error

This means invalid data is detected at the entry point instead of causing an error later inside the application.

Why Normal Python Type Hints Are Not Enough

Python supports type hints.

For example, a function parameter can be declared as an integer or string.

However, type hints mainly describe what type of value is expected. They do not automatically provide complete runtime data validation.

Pydantic uses type annotations to perform actual runtime validation and parsing.

This is one of the main reasons Pydantic is useful.

The Problem Without Data Validation

Suppose an application expects a person's age to be an integer.

However, external data sends text such as:

"hello"

instead of a valid number.

Without proper validation, the bad value may enter the application and cause an error much later during calculation or business logic.

The problem is:

Bad Data Enters



Application Accepts It



Several Operations Happen



Program Fails Later



Debugging Becomes Difficult

Pydantic solves this problem by checking the data immediately.

Data Validation

Validation means checking whether incoming data satisfies the expected requirements.

For example, an application may require:

- name to be a string
- age to be an integer
- price to be a float
- active to be a boolean

Pydantic checks these requirements when data is processed by the model.

Data Parsing

Parsing means taking incoming data and converting it into a structured form that the application can use.

For example, data may arrive as a dictionary from an API request. Pydantic can process that data and create a structured Python model object.

Type Coercion

By default, Pydantic can convert many compatible values into the expected data type.

For example:

"25" → 25

Here:

Input Type : String

Expected Type : Integer

Final Type : Integer

This process is called **type coercion**.

However, conversion only works when the value can reasonably be converted.

For example:

"25" → Integer conversion possible

"hello" → Integer conversion impossible

If conversion is impossible, Pydantic raises a validation error.

ValidationError

When incoming data does not satisfy the model requirements, Pydantic raises a validation error.

A useful validation error can tell us:

- Which field failed
- What kind of validation failed
- What input caused the problem
- What type of value was expected

This makes debugging easier.

Pydantic and FastAPI

Pydantic is commonly used with FastAPI, but Pydantic is an independent Python library.

It can be used with:

- FastAPI
- Django
- Flask
- AI applications
- Machine learning pipelines
- Data processing systems
- Automation scripts
- CLI applications
- Configuration systems
- JSON validation systems

FastAPI uses Pydantic extensively for request and response data validation, but Pydantic can be used without FastAPI.

Why Pydantic is Important in AI Applications

Modern AI applications often exchange structured data between multiple components.

For example:

User Input



API



AI Model



Structured Response



Database

Incorrect data at any stage can create problems.

Pydantic can be used to validate:

- User input
- API request data
- AI model output
- Structured LLM responses
- Agent tool arguments
- Configuration values
- Database input
- JSON responses

This makes the flow of data more reliable and predictable.

Important Points

1. Pydantic is used for runtime data validation.
 2. Pydantic uses Python type annotations to define expected data structures.
 3. It validates incoming data before the application depends on it.
 4. It can convert compatible input values into expected data types.
 5. This automatic conversion is called type coercion.
 6. Invalid data produces clear validation errors.
 7. Pydantic models are generally created by inheriting from BaseModel.
 8. Pydantic is not limited to FastAPI.
 9. It can be used in web development, AI systems, data pipelines, scripts, and configuration management.
 10. The main goal is to create trusted and predictable application data.
-

Syntax

Basic Pydantic model syntax:

Import BaseModel from Pydantic

Create a class

Inherit the class from BaseModel

Define:

field name

field type

Create the model object with input data

Pydantic validates the data

If valid:

Model object is created

If invalid:

Validation error is raised

General structure:

BaseModel

↓

Custom Model

↓

Fields + Type Annotations

↓

Incoming Data

↓

Validation and Parsing

↓

Trusted Model Object